

MTD 367 iOS Application Development

- ~~Lesson #1 - Building Apps That Matter and Swift~~
- ~~Lesson #2 - Dealing With Swift Types And SwiftUI~~
- ~~Lesson #3 - Doubling Down On SwiftUI~~
- **Lesson #4 - All To-Do With SwiftData (Today)**
- Lesson #5 - Streaming Multimedia Swiftly (8/9/2025)
- Lesson #6 - Putting The Map On Your App (15/9/2025)

? Questions About Lesson #3

-  More SwiftUI Features
 - Views: `List`, `TextField`, `NavigationStack`
 - View Modifiers: Gestures, Animations
-  Setting up a Project
-  Testing in simulator

? Questions About Virtual Lab #3?

Keep improving your Blackjack App!

Today's Agenda

- ☐ Basics of SwiftData

Did You Notice?

- Right now, our **Blackjack app is not persistent.**
- Every time you launch it:
 - Player hands reset
 - No memory of game history
- We want to **save and restore app data** even after closing the app.

Why Persist Data?

- Users expect apps to **remember things**.
- Examples:
 - User scores
 - Game progress
 - Settings and preferences
- Data persistence improves:
 - User experience
 - Reliability
 - Trust in your app

What Is SwiftData?

- **New persistence framework** introduced in iOS 17.
- Declarative and modern:
 - Works seamlessly with SwiftUI
- Inspired by Core Data but:
 - Less boilerplate
 - Swift-native syntax
 - Easier to learn!

How SwiftData Works?

- You define your data using `@Model`.
- SwiftData automatically:
 - Stores data to disk
 - Handles saving and loading
- Integrates beautifully with SwiftUI:
 - Use `@Query` to fetch data into views

SwiftData Vs Core Data

Feature	Core Data	SwiftData
Syntax	Objective-C flavoured	Pure Swift
Integration	UIKit and SwiftUI	Designed for SwiftUI
Boilerplate	More code	Less code
Learning Curve	Steeper	Easier

SwiftData Vs Core Data Continued

Feature	Core Data	SwiftData
Underlying Technology	Core Data Engine	Built on Core Data but modernized
SwiftUI Integration	Requires adapters	Native support
Type Safety	Lower	Strong type safety

XCode



Hands-On #5

Build a simple To-Do List app that:

- Let Users add tasks
- Displays tasks in a list
- Make sure data is persistent

✓ Summary

- ✓ Basics Of SwiftData
 - Why Persist data?
 - What Is SwiftData?
 - How SwiftData Works?
 - SwiftData Vs CoreData

Virtual Lab #4

Improve your To-Do List app!

- Use a gesture to mark a task as done
- Display done in a separate list
- Display done and in-progress tasks separately
- Use a gesture to switch views between done and in-progress

? Questions?

I'd love to hear how the class went or how I can improve.

Feel free to email me at:

muhammadfadzuli001@suss.edu.sg

 All course materials are available at:

www.github.com/iluzdaf/mtd367

Appendix

Defining a Model

SwiftData takes care of persistence for you!

```
@Model
class Player {
    var name: String
    var score: Int

    init(name: String, score: Int) {
        self.name = name
        self.score = score
    }
}
```

Fetching Data

No need for complex fetch requests!

```
@Query var players: [Player]

var body: some View {
    List(players) { player in
        Text(player.name)
    }
}
```

Saving Data

Saves your model to persistent storage.

```
let context = ModelContext(container)
let player = Player(name: "Alice", score: 10)

context.insert(player)

try context.save()
```